Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №2 по курсу

«Операционные системы»

Группа: М8О-212БВ-25

Студент: Волгин М.Е

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 01.11.26

Москва, 2026

Постановка задачи

Вариант 2.

Составить программу на языке Си, обрабатывающую данные в многопоточном режиме. При обработки использовать стандартные средства создания потоков операционной системы (Windows/Unix). Ограничение максимального количества потоков, работающих в один момент времени, должно быть задано ключом запуска вашей программы. Так же необходимо уметь продемонстрировать количество потоков, используемое вашей программой с помощью стандартных средств операционной системы.
В отчете привести исследование зависимости ускорения и эффективности алгоритма от входных данных и количества потоков. Получившиеся результаты необходимо объяснить.
Отсортировать массив целых чисел при помощи параллельного алгоритма быстрой сортировки

Общий метод и алгоритм решения

В лабораторной работе реализована параллельная быстрая сортировка (Parallel QuickSort) на языке C с использованием библиотеки POSIX Threads (`pthread.h`).

Реализованы две версии алгоритма:

- Последовательная — сортировка выполняется одним потоком.
- Параллельная — массив разбивается на части, которые могут сортироваться одновр еменно разными потоками.

Используется модель Fork & Join: создаём новый поток (`pthread_create`), выполняем рабо ту параллельно и ожидаем его завершения (`pthread_join`).

Алгоритм решения

1. Создаём массив из `500000` целых чисел и заполняем его псевдослучайными значени ями.
2. Выполняем последовательную сортировку `quick_sort()` и замеряем время.
3. Заново заполняем массив теми же числами для честного сравнения.
4. Запускаем `quick_sort_parallel()`.
5. Функция `partition()` выбирает опорный элемент и разделяет массив на две части.
6. Если левая часть достаточно большая (`4096` элементов и более) и лимит потоков не достигнут, через `pthread_create()` создаётся новый поток для её сортировки.
7. Основной поток сортирует правую часть, после чего через `pthread_join()` ожидает з авершения созданного потока.
8. Если создать поток нельзя, обе части сортируются последовательно текущим поток ом.
9. Проверяем правильность сортировки, измеряем время и сравниваем результаты.

Код программы

main.c

```c
#include <unistd.h>
#include <time.h>
#include <pthread.h>

#define ARRAY_SIZE 500000
#define MAX_THREAD_LIMIT 64
#define PARALLEL_THRESHOLD 4096

static int values[ARRAY_SIZE];

static pthread_mutex_t thread_count_mutex = PTHREAD_MUTEX_INITIALIZER;

static int active_threads = 1;
static int thread_limit = 1;

struct sort_task {
    int *array;
    int low;
    int high;
};

static void quick_sort_parallel(int array[], int low, int high);
static int reserve_thread_slot(void);
static void release_thread_slot(void);

static int partition(int array[], int low, int high) {
    int pivot = array[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (array[j] <= pivot) {
            i++;


            int temp = array[i];
            array[i] = array[j];
            array[j] = temp;
        }
    }

    int temp = array[i + 1];
    array[i + 1] = array[high];
    array[high] = temp;

    return i + 1;
}


void quick_sort(int array[], int low, int high) {
    if (low < high) {
        int pivot_index = partition(array, low, high);

        quick_sort(array, low, pivot_index - 1);
```

```
        quick_sort(array, pivot_index + 1, high);
    }
}

static void* sort_worker(void* arg) {
    struct sort_task *task = arg;

    quick_sort_parallel(task->array, task->low, task->high);

    return 0;
}

static void quick_sort_parallel(int array[], int low, int high) {
    if (low >= high) {
        return;
    }

    int pivot_index = partition(array, low, high);

    if (pivot_index - low >= PARALLEL_THRESHOLD && reserve_thread_slot()) {

        struct sort_task left_task = {
            array, low, pivot_index - 1
        };

        pthread_t worker_thread;

        int error = pthread_create(
            &worker_thread, 0, sort_worker, &left_task
        );

        if (error == 0) {
            quick_sort_parallel(array, pivot_index + 1, high);

            error = pthread_join(worker_thread, 0);

            if (error != 0) {
                const char message[] = "pthread_join failed\n";
                (void)write(
                    STDERR_FILENO, message, sizeof(message) - 1
                );
                _exit(1);
            }

            release_thread_slot();
            return;
        }

        release_thread_slot();
    }

    quick_sort_parallel(array, low, pivot_index - 1);
    quick_sort_parallel(array, pivot_index + 1, high);

}
```

```
static int reserve_thread_slot(void) {

    int reserved = 0;

    if (pthread_mutex_lock(&thread_count_mutex) != 0) {
        _exit(1);
    }

    if (active_threads < thread_limit) {
        active_threads++;
        reserved = 1;
    }

    if (pthread_mutex_unlock(&thread_count_mutex) != 0) {
        _exit(1);
    }


    return reserved;
}

static void release_thread_slot(void) {
    int error = pthread_mutex_lock(&thread_count_mutex);

    if (error != 0) {
        _exit(1);
    }

    active_threads--;

    error = pthread_mutex_unlock(&thread_count_mutex);

    if (error != 0) {
        _exit(1);
    }
}


static void fill_array(int array[], int length) {
    unsigned int state = 123456789u;

    for (int i = 0; i < length; i++) {
        state = state * 1664525u + 1013904223u;
        array[i] = (int)(state % 1000000u);
    }
}

static int is_sorted(const int array[], int length) {
    for (int i = 1; i < length; i++) {
        if (array[i - 1] > array[i]) {
            return 0;
        }
    }


    return 1;
}
```

```c
static void write_unsigned(unsigned long long value) {
    char digits[20];
    int count = 0;

    do {
        digits[count++] = (char)('0' + value % 10ULL);
        value /= 10ULL;
    } while (value != 0ULL);

    while (count > 0) {
        count--;
        (void)write(STDOUT_FILENO, &digits[count], 1);
    }


}

int parse_thread_limit(const char* text, int* result) {
    if (text[0] == '\0') {
        return 0;
    }

    int value = 0;

    for (int i = 0; text[i] != '\0'; i++) {
        if (text[i] < '0' || text[i] > '9') {
            return 0;
        }

        int digit = text[i] - '0';

        if (value > (MAX_THREAD_LIMIT - digit) / 10) {
            return 0;
        }


        value = value * 10 + digit;

    }

    if (value < 1) {
        return 0;
    }

    *result = value;
    return 1;
}

int main(int argc, char* argv[]) {
    int max_threads = 0;

    if (argc != 2 || !parse_thread_limit(argv[1], &max_threads)) {
        const char message[] = "Usase: ./lab 1...64\n";
        (void)write(STDERR_FILENO, message, sizeof(message) - 1);
```

```
    return 1;
}

thread_limit = max_threads;

fill_array(values, ARRAY_SIZE);

struct timespec start;
struct timespec end;

if (clock_gettime(CLOCK_MONOTONIC, &start) != 0) {
    const char message[] = "Start timer fail\n";
    (void)write(STDOUT_FILENO, message, sizeof(message) - 1);
    return 1;
}

quick_sort(values, 0, ARRAY_SIZE - 1);

if (clock_gettime(CLOCK_MONOTONIC, &end) != 0) {
    const char message[] = "Stop timer fail\n";
    (void)write(STDOUT_FILENO, message, sizeof(message) - 1);
    return 1;
}

long long seconds = (long long)end.tv_sec - (long long)start.tv_sec;

long long nanoseconds = (long long)end.tv_nsec - (long long)start.tv_nsec;

long long elapsed_ns = seconds * 1000000000LL + nanoseconds;

if (elapsed_ns < 0) {
    const char message[] = "not valid time\n";
    (void)write(STDOUT_FILENO, message, sizeof(message) - 1);
    return 1;
}

const char size_message[] = "Size: ";
(void)write(STDOUT_FILENO, size_message, sizeof(size_message) - 1);

write_unsigned(ARRAY_SIZE);

const char elements_message[] = " elements\n";
(void)write(STDOUT_FILENO, elements_message, sizeof(elements_message) - 1);

const char time_message[] = "Sequential QuickSort time: ";
(void)write(STDOUT_FILENO, time_message, sizeof(time_message) - 1);

write_unsigned((unsigned long long)(elapsed_ns / 1000000000LL));

const char dot[] = ".";
(void)write(STDOUT_FILENO, dot, sizeof(dot) - 1);

unsigned long long fraction = (unsigned long long)((elapsed_ns % 1000000000LL) / 1000LL);

unsigned long long divisor = 100000ULL;

for (int i = 0; i < 6; i++) {
```

```
    char digit = (char)('0' + (fraction / divisor) % 10ULL);
    (void)write(STDOUT_FILENO, &digit, 1);
    divisor /= 10ULL;
}

const char seconds_message[] = " seconds\n";
(void)write(STDOUT_FILENO, seconds_message, sizeof(seconds_message) - 1);

if (!is_sorted(values, ARRAY_SIZE)) {
    const char message[] = "Sorting verification failed\n";
    (void)write(STDOUT_FILENO, message, sizeof(message) - 1);
    return 1;
}

const char success_message[] = "Sorted correctly: yes\n";
(void)write(STDOUT_FILENO, success_message, sizeof(success_message) - 1);



fill_array(values, ARRAY_SIZE);

if (clock_gettime(CLOCK_MONOTONIC, &start) != 0) {
    const char message[] = "Timer start failed\n";
    (void)write(STDERR_FILENO, message, sizeof(message) - 1);
    return 1;
}


active_threads = 1;

quick_sort_parallel(values, 0, ARRAY_SIZE - 1);

if (clock_gettime(CLOCK_MONOTONIC, &end) != 0) {
    const char message[] = "Timer stop failed\n";
    (void)write(STDERR_FILENO, message, sizeof(message) - 1);
    return 1;
}

seconds = (long long)end.tv_sec - (long long)start.tv_sec;

nanoseconds = (long long)end.tv_nsec - (long long)start.tv_nsec;

elapsed_ns = seconds * 1000000000LL + nanoseconds;

if (elapsed_ns < 0) {
    const char message[] = "Invalid elapsed time\n";
    (void)write(STDERR_FILENO, message, sizeof(message) - 1);
    return 1;
}

const char selected_time_message[] = "QuickSort time with thread limit ";

(void)write(STDOUT_FILENO, selected_time_message, sizeof(selected_time_message) - 1);

write_unsigned((unsigned long long)max_threads);

const char separator[] = ": ";
```

```c
    (void)write(STDOUT_FILENO, separator, sizeof(separator) - 1);

    write_unsigned((unsigned long long)(elapsed_ns / 1000000000LL));

    const char parallel_dot[] = ".";
    (void)write(STDOUT_FILENO, parallel_dot, sizeof(parallel_dot) - 1);

    fraction = (unsigned long long) ((elapsed_ns % 1000000000LL) / 1000LL);

    divisor = 100000ULL;

    for (int i = 0; i < 6; i++) {
        char digit = (char)('0' + (fraction / divisor) % 10ULL);
        (void)write(STDOUT_FILENO, &digit, 1);
        divisor /= 10ULL;
    }

    const char parallel_seconds_message[] = " seconds\n";
    (void)write(STDOUT_FILENO, parallel_seconds_message, sizeof(parallel_seconds_message) - 1);

    if (!is_sorted(values, ARRAY_SIZE)) {
        const char message[] = "Parallel sorting fail\n";
        (void)write(STDERR_FILENO, message, sizeof(message) - 1);
        return 1;
    }

    const char parallel_success_message[] = "Sorting verification passed: yes\n";

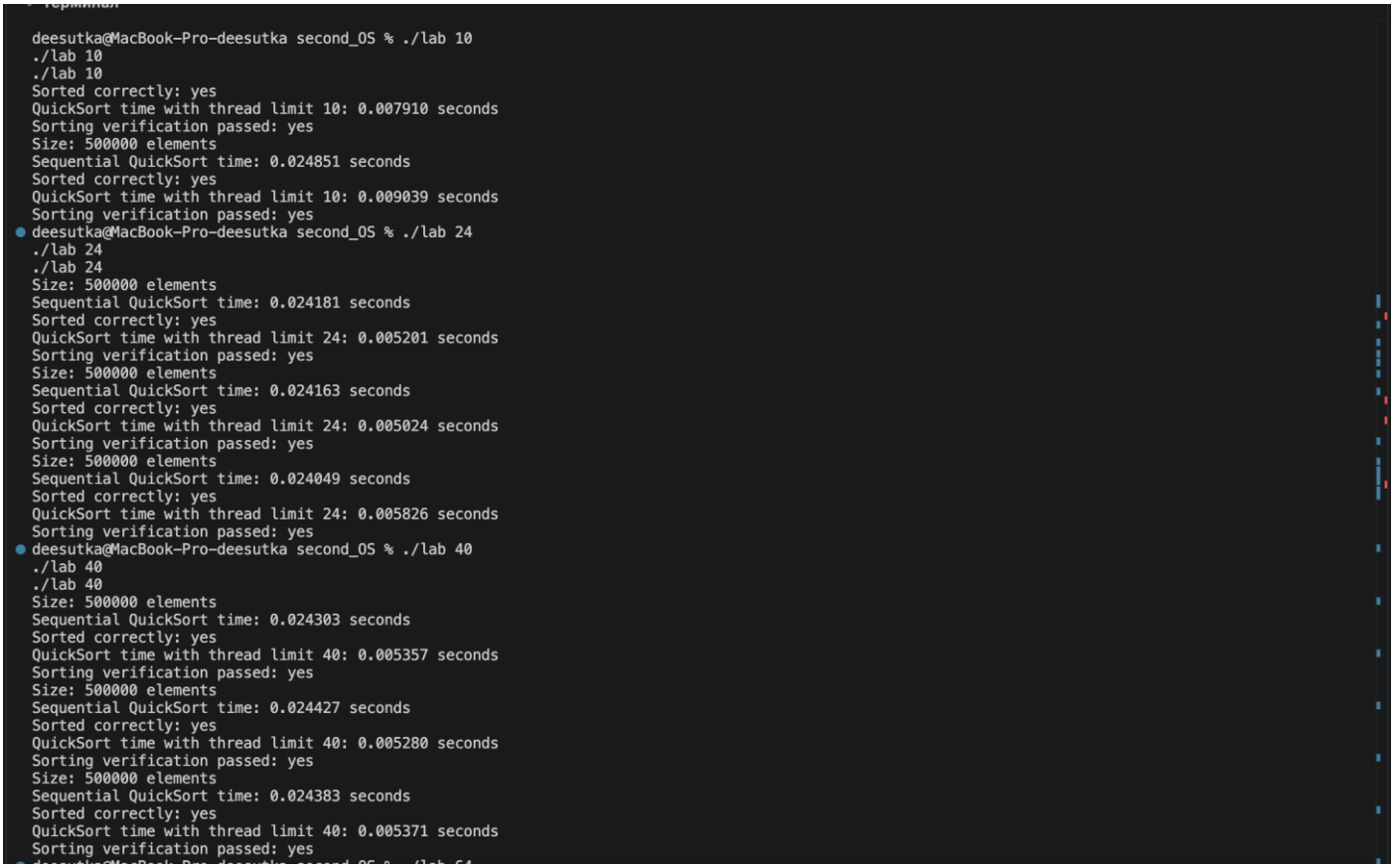
    (void)write(STDOUT_FILENO, parallel_success_message, sizeof(parallel_success_message) - 1);

    return 0;
}
```

Протокол работы программы

```
deesutka@MacBook-Pro-deesutka second_OS % gcc -O2 -pthread main.c -o lab
deesutka@MacBook-Pro-deesutka second_OS % nproc
zsh: command not found: nproc
deesutka@MacBook-Pro-deesutka second_OS % sysctl -n hw.logicalcpu
10
deesutka@MacBook-Pro-deesutka second_OS % ./lab 1
./lab 1
./lab 1
Size: 500000 elements
Sequential QuickSort time: 0.029100 seconds
Sorted correctly: yes
QuickSort time with thread limit 1: 0.027313 seconds
Sorting verification passed: yes
Size: 500000 elements
Sequential QuickSort time: 0.027501 seconds
Sorted correctly: yes
QuickSort time with thread limit 1: 0.027025 seconds
Sorting verification passed: yes
Size: 500000 elements
Sequential QuickSort time: 0.026509 seconds
Sorted correctly: yes
QuickSort time with thread limit 1: 0.026206 seconds
Sorting verification passed: yes
deesutka@MacBook-Pro-deesutka second_OS % ./lab 2
./lab 2
./lab 2
Size: 500000 elements
Sequential QuickSort time: 0.024283 seconds
Sorted correctly: yes
QuickSort time with thread limit 2: 0.021228 seconds
Sorting verification passed: yes
Size: 500000 elements
Sequential QuickSort time: 0.025023 seconds
Sorted correctly: yes
QuickSort time with thread limit 2: 0.021188 seconds
Sorting verification passed: yes
Size: 500000 elements
Sequential QuickSort time: 0.025733 seconds
Sorted correctly: yes
QuickSort time with thread limit 2: 0.021329 seconds
Sorting verification passed: yes
deesutka@MacBook-Pro-deesutka second_OS % ./lab 4
./lab 4
./lab 4
Size: 500000 elements
Sequential QuickSort time: 0.023959 seconds
Sorted correctly: yes
QuickSort time with thread limit 4: 0.015035 seconds
Sorting verification passed: yes
Size: 500000 elements
```

```
./lab 2
./lab 2
deesutka@MacBook-Pro-deesutka second_OS % ./lab 4
./lab 4
./lab 4
Size: 500000 elements
Sequential QuickSort time: 0.023959 seconds
Sorted correctly: yes
QuickSort time with thread limit 4: 0.015035 seconds
Sorting verification passed: yes
Size: 500000 elements
Sequential QuickSort time: 0.024534 seconds
Sorted correctly: yes
QuickSort time with thread limit 4: 0.014192 seconds
Sorting verification passed: yes
Size: 500000 elements
Sequential QuickSort time: 0.024709 seconds
Sorted correctly: yes
QuickSort time with thread limit 4: 0.014918 seconds
Sorting verification passed: yes
deesutka@MacBook-Pro-deesutka second_OS % ./lab 8
./lab 8
./lab 8
Size: 500000 elements
Sequential QuickSort time: 0.034262 seconds
Sorted correctly: yes
QuickSort time with thread limit 8: 0.009549 seconds
Sorting verification passed: yes
Size: 500000 elements
Sequential QuickSort time: 0.024910 seconds
Sorted correctly: yes
QuickSort time with thread limit 8: 0.007696 seconds
Sorting verification passed: yes
Size: 500000 elements
Sequential QuickSort time: 0.036123 seconds
Sorted correctly: yes
QuickSort time with thread limit 8: 0.011632 seconds
Sorting verification passed: yes
deesutka@MacBook-Pro-deesutka second_OS % ./lab 10
./lab 10
./lab 10
Size: 500000 elements
Sequential QuickSort time: 0.024417 seconds
Sorted correctly: yes
QuickSort time with thread limit 10: 0.008090 seconds
Sorting verification passed: yes
Size: 500000 elements
Sequential QuickSort time: 0.024731 seconds
Sorted correctly: yes
QuickSort time with thread limit 10: 0.007910 seconds
Sorting verification passed: yes
Size: 500000 elements
```





итоговая_таблица

Максимум потоков	Медиана последовательной сортировки (мс)	Медиана параллельной сортировки (мс)	Ускорение	Эффективность (%)
1	24.791	27.025	0.917	91.7
2	24.791	21.228	1.168	58.4
4	24.791	14.918	1.662	41.5
8	24.791	9.549	2.596	32.5
10	24.791	8.090	3.064	30.6
24	24.791	5.201	4.767	19.9
40	24.791	5.357	4.628	11.6
64	24.791	6.408	3.869	6.0

Ускорение (S) — метрика, показывающая, во сколько раз параллельный алгоритм выполняется быстрее последовательного.

Формула: **S = Ts / Tp**

Эффективность (E) — метрика, показывающая эффективность использования выделенных потоков.

Формула: **E = (S / p) × 100%**

Где:

- **Ts** — время выполнения последовательного алгоритма (мс).
- **Tp** — время выполнения параллельного алгоритма (мс).
- **p** — количество потоков (в данной работе — установленный лимит).
- **S** — ускорение алгоритма.
- **E** — эффективность параллельного алгоритма в процентах.

Чем выше ускорение, тем быстрее параллельная реализация относительно последовательной. Чем выше эффективность, тем больше ускорение в расчёте на один поток.

Вывод

В ходе лабораторной работы были реализованы и исследованы последовательный и параллельный алгоритмы быстрой сортировки с использованием POSIX Threads.

По результатам экспериментов установлено, что увеличение количества потоков первоначально сокращает время сортировки. Наилучший результат получен при лимите **24 потока**: время выполнения составило **5,201 мс**, ускорение — **4,767 раза**, эффективность — **19,9%**.

При дальнейшем увеличении лимита до 40 и 64 потоков время выполнения возрастает, а эффективность снижается. Это объясняется накладными расходами на создание и синхронизацию потоков, а также ограниченными вычислительными ресурсами процессора.

Построенные графики подтверждают зависимость времени выполнения, ускорения и эффективности от количества потоков.

Таким образом, **многопоточность позволяет значительно ускорить сортировку**, однако чрезмерное увеличение количества потоков не всегда приводит к повышению производительности.